

Ministry of Education of Republic of Moldova
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics

Laboratory work 4: The xv6 shell from outside

Author:
st. gr. FAF-241 Mitrofan Alexia
Verified:
asist. univ. Reițman Patricia

Chișinău - 2026

Contents

Objectives **2**

Part 1. Fixing sleep for xv6 **3**

Part 2. bash versus xv6 shell **5**

Part 3. Printing argv **11**

Part 4. Reading the shell **15**

CONCLUSION **17**

REFERENCES **18**

OBJECTIVES

The aim of this laboratory work is to discover and work with the xv6 shell, and use it side by side with Linux shell and compare what functionalities each shell has, what tasks belong to it, and what tasks belong to the kernel and/or the program itself. We will also create a program that prints the arguments it receives, which shows exactly who expands and who removes quotes, and study the differences.

Part 1. Fixing sleep for xv6

```
cd ~/xv6-riscv
grep -n "pause\|sleep" user/user.h
```

```
vboxuser@Ubuntu2604:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
vboxuser@Ubuntu2604:~/xv6-riscv$
```

Figure 1.1: Checking the name my tree uses

```
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
 if(argc != 2) {
   fprintf(2, "usage: sleep <ticks>\n");
   exit(1);
   }
 pause(atoi(argv[1]));  // was sleep() before Aug 2025
 exit(0);
}
```

Open

sleep.c
~/xv6-riscv/user

```
#include "kernel/types.h"
#include "user/user.h"

int
main (int argc, char *argv[])
{
 if (argc != 2) {
   fprintf(2, "usage: sleep <ticks>\n");
   exit(1);
  }
  pause(atoi(argv[1])); // system call into kernel
  exit(0);
}
```

Figure 1.2: The program fixed (I fixed it back during lab 2)

```
make qemu
$ sleep 20
$ sleep
usage: sleep <ticks>
```

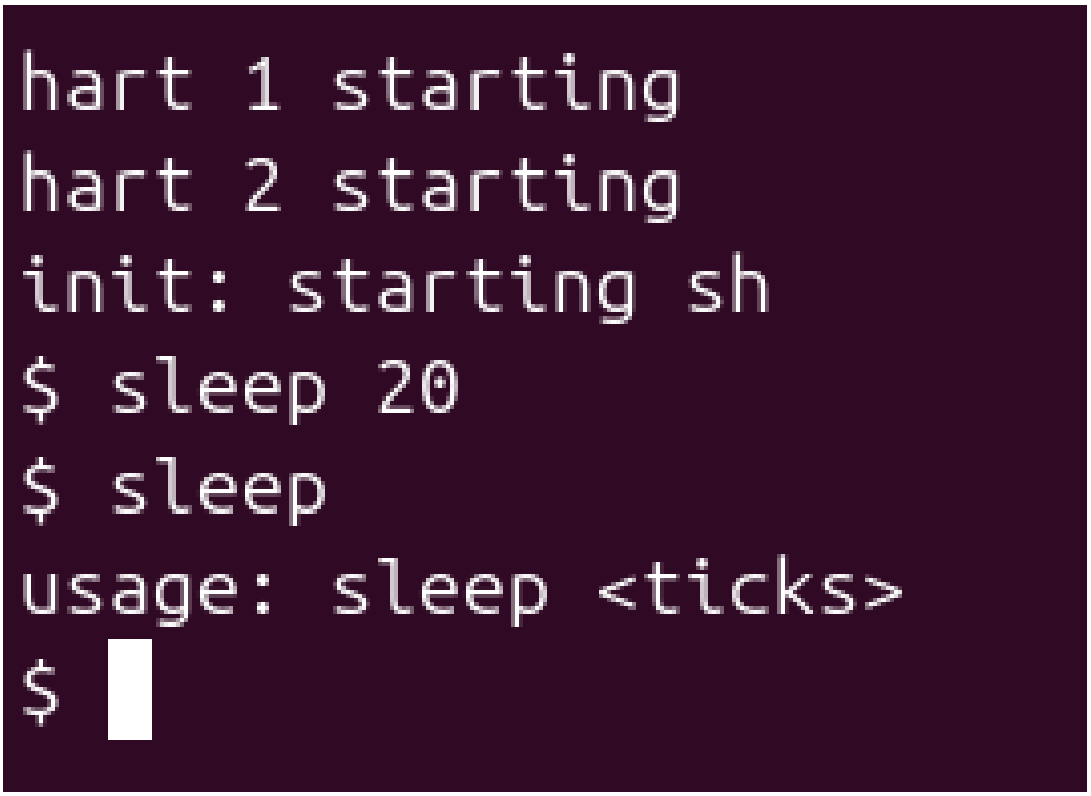


Figure 1.3: Running the program

Part 2. bash versus xv6 shell

Command	Tests	bash	xv6
`echo hello > f ; cat < f`	redirection	prints `hello` (written to `f`, then read back by `cat`)	`exec echo failed`; `f` is created but empty, so `cat` prints nothing
`ls | grep READ`	pipe	`README` (match highlighted)	`README 2 2 2441`
`(echo a; echo b) | wc`	grouping, list	`2 2 4`	`2 2 4`
`echo one ; echo two`	sequence	`one` then `two`	`one` then `two`
`echo bg &`	background	prints job number `[1] 6789`, then `bg`, then `[1]+ Done echo bg`	prints `bg` only; it takes `bg &` as regular argument
`ls *`	globbing	`README f` (`*` expanded to file names)	`ls: cannot open *` (`*` passed literally)
`echo $HOME`	variables	`/home/vboxuser`	`$HOME` (not expanded)
`echo "a b"`	quotes	`a b` (quotes removed)	`"a b"` (quotes kept, printed literally)
`mkdir d ; cd d ; ls ..`	built-in `cd`	`README d f` (`cd` worked, so `..` is the parent of `d`)	`exec cd failed`; the shell stayed in root, so `ls ..` listed the root directory
`arrow up, Tab, Ctrl+R`	history, completion	we can search previous work in `reverse-i-search`)	no history, completion or search. we just move up the output we were given

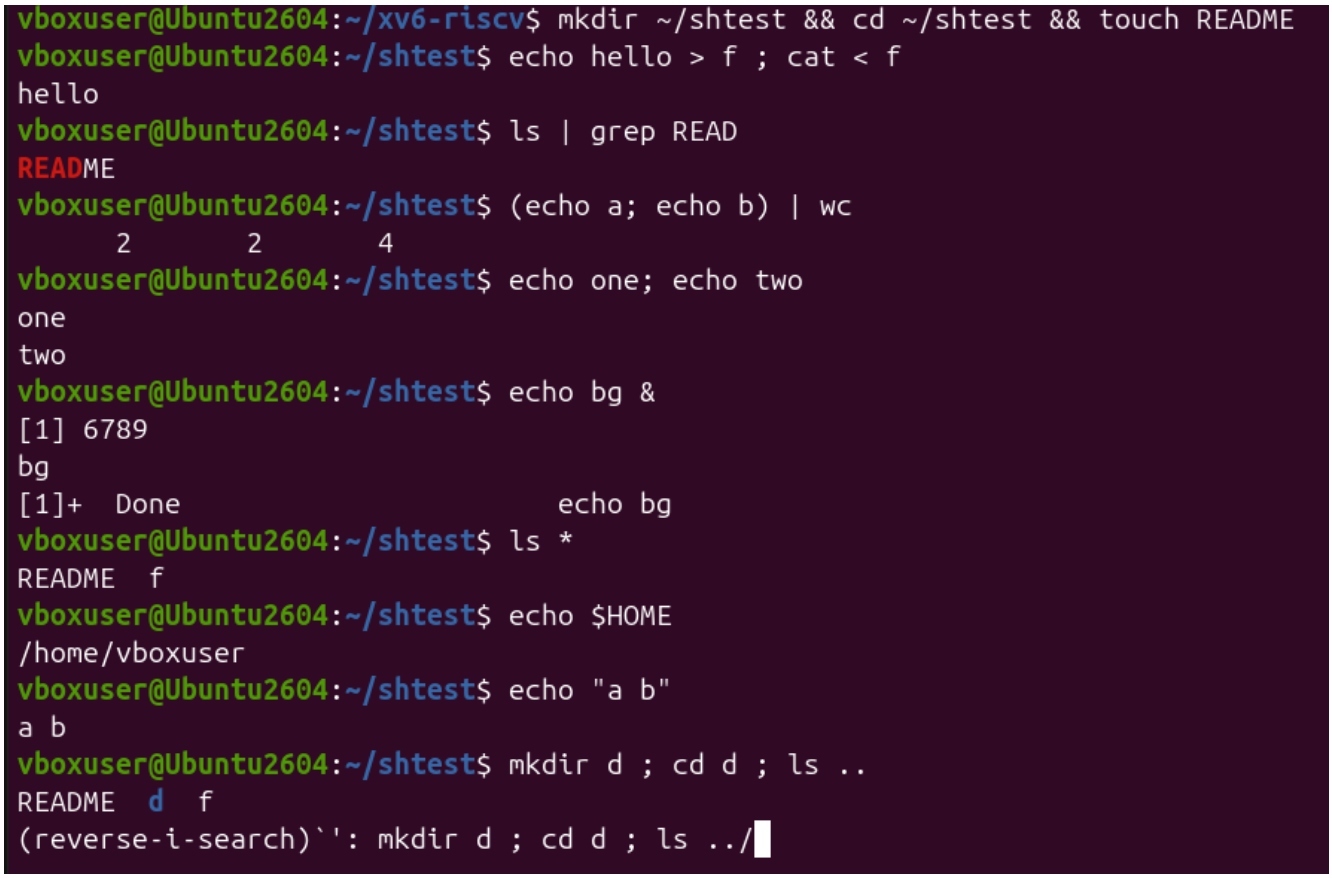


Figure 2.1: Commands run in bash

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ echo hello > f ; cat < f
exec echo failed
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
```

Figure 2.2: Commands run in xv6 shell (pt. 1)

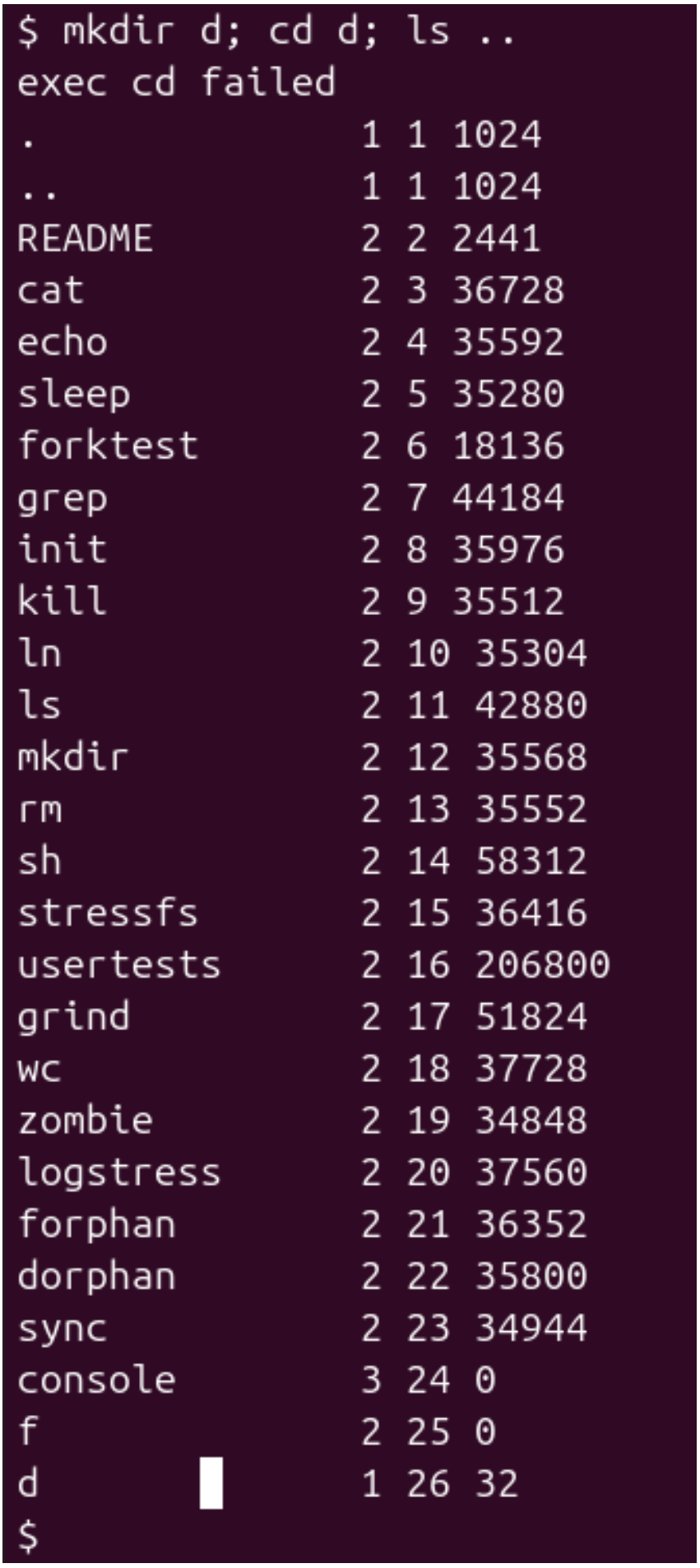


Figure 2.3: Commands run in xv6 shell (pt. 2)

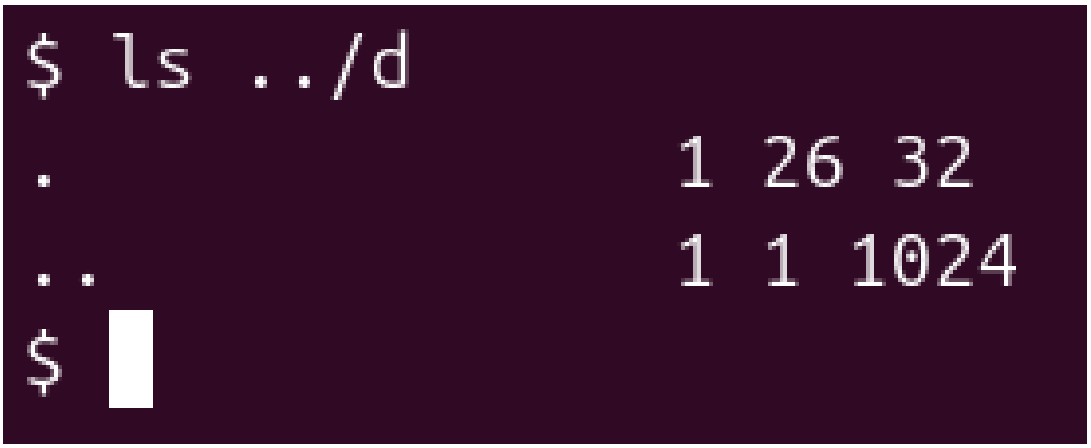


Figure 2.4: Working alternative 1 for ls



Figure 2.5: Working alternative 2 for ls

1. **List the features both shells have, and the ones only bash has.**
   Both shells have the following features: running programs with `exec`, input/output redirection (`<`, `>`), pipes (`|`), sequencing (`;`), grouping with parentheses, background execution (`&`), and the built-in `cd` (in xv6 only when it is the first command on the line).
   The following features are only in Linux shell: globbing (`*.c`), variable expansion (`$HOME`), quoting (single and double quotes), job control messages for background jobs (`[1] 6789`, `Done`), a `PATH` for finding programs, command history (arrow up, `Ctrl+R`) and tab completion.

2. **In xv6, ls stops working after cd d. Why? Find a way to run ls from inside d anyway. (Hint: where do the xv6 programs live?)**
   ls stop working because xv6 has no $PATH. When typing ls, the shell only looks for a file named ls in the current directory. The programs live in the root directory, so once we cd d, there's no ls in the directory we're standing in, and exec fails. We can

run it anyway by giving the shell a path to where the program lives: /ls (absolute) or ../ls (relative, since d sits directly under the root we are currently in).

3. **Which missing feature would you miss most in daily work, and which would be easiest to add?**
   If I were to choose one, I would add the ability of double quotes to interpolate variables. I think the easiest one to add is to make $HOME print the actual folder.

Part 3. Printing argv

```c
#include "kernel/types.h"
#include "user/user.h"
int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
exit(0);
}
```

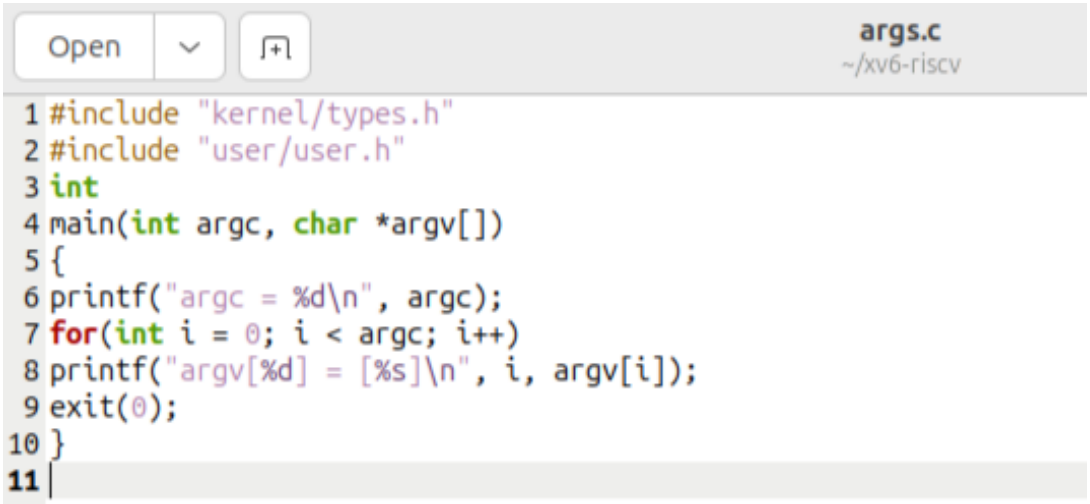


Figure 3.1: args.c

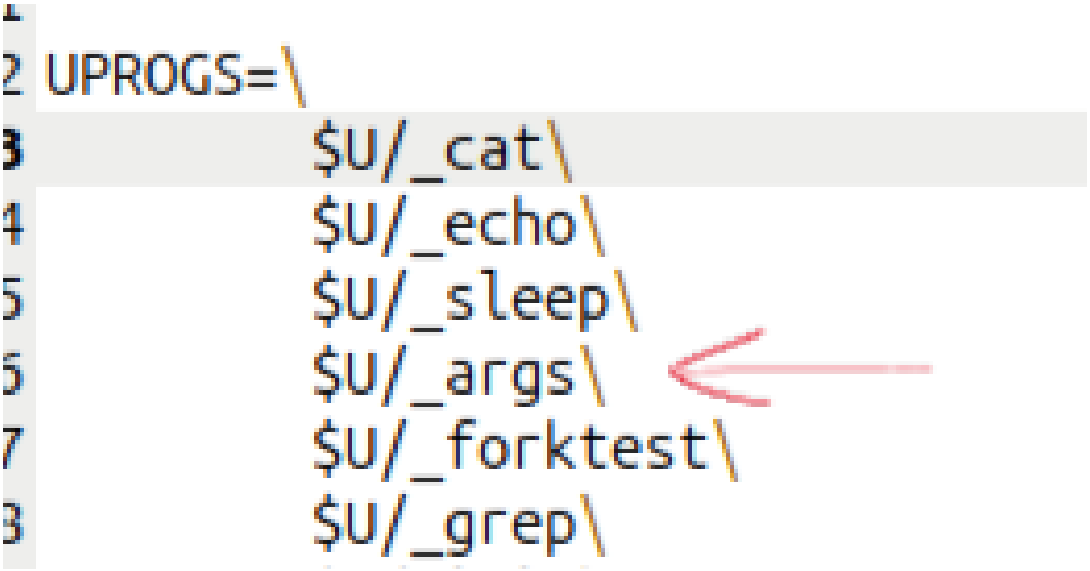


Figure 3.2: UPROGS line

```
args one  two  three
args *.c "x y"
args $HOME
args 'single quoted'
```

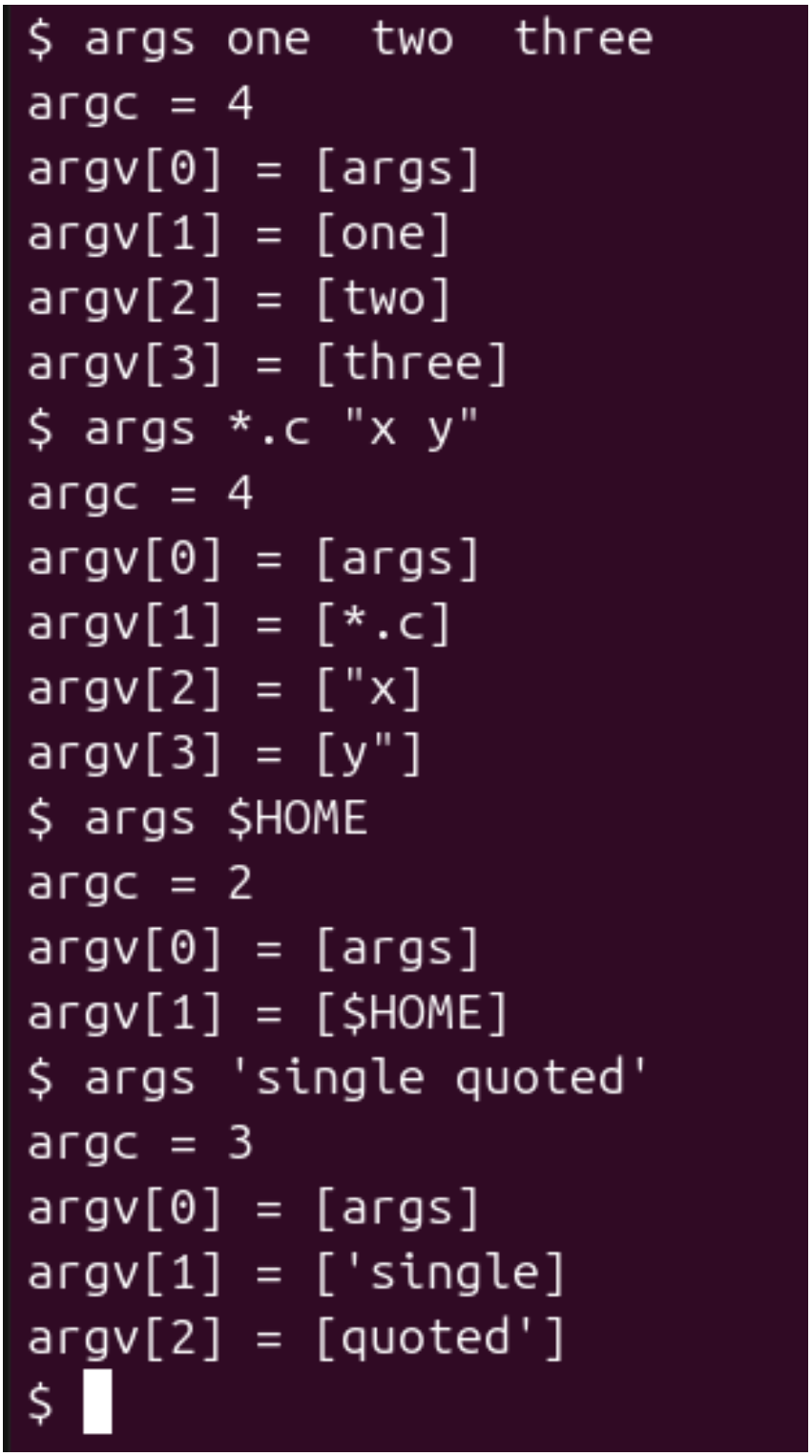


Figure 3.3: args.c output in xv6 shell

```
vboxuser@Ubuntu2604:~/xv6-riscv$ ./args one two three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
vboxuser@Ubuntu2604:~/xv6-riscv$ ./args one  two  three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
vboxuser@Ubuntu2604:~/xv6-riscv$ ./args *.c "x y"
argc = 3
argv[0] = [./args]
argv[1] = [args_linux.c]
argv[2] = [x y]
vboxuser@Ubuntu2604:~/xv6-riscv$ args $HOME
Command 'args' not found, but can be installed with:
sudo apt install ucommon-utils
vboxuser@Ubuntu2604:~/xv6-riscv$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/vboxuser]
vboxuser@Ubuntu2604:~/xv6-riscv$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.4: args_linux output in bash

1. **Paste both outputs for `args *.c "x y"` . How many arguments did each program receive?**

   In xv6 it's the following:

```
1    argc = 4
2    argv[0] = [args]
3    argv[1] = [*.c]
4    argv[2] = ["x]
5    argv[3] = [y"]
```

   In the Linux shell it's the following:

```
1    argc = 3
2    argv[0] = [./args]
3    argv[1] = [args_linux.c]
```

```
4     argv[2] = [x y]
```

xv6 receives 4 arguments, Linux shell receives 3. If we don't count the program name (argv[0]) as an argument, then the answer would be 3 for xv6 and 2 for Linux. It's this way because Linux does more than xv6: for the argument `*.c`, Linux does globbing (aka it finds any .c files), while xv6 takes it as normal text and prints it as it is. Then, with "x y", Linux removes the double quotes and counts it as a single argument, but xv6 doesn't handle quotes so it takes "x y" as two separate arguments: "x" and "y".

2. **So who turns *.c into file names and removes the quotes: the program, the kernel, or the shell?**
   The shell turns *.c into files names and removes the quotes. The kernel only copies the the argument list, and the program just receives the original text.

3. **What happened to the extra spaces in `one two three` ?**
   The extra spaces were discarded, because the shell splits the line into words wherever it finds whitespace, and any amount of space presses only counts as one separator.

Part 4. Reading the shell

```
wc -l user/sh.c
grep -n "#define" user/sh.c    # the kinds of commands
grep -n "case " user/sh.c      # where each  kind is run
grep -n "cd" user/sh.c | head
```

```
vboxuser@Ubuntu2604:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
vboxuser@Ubuntu2604:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
vboxuser@Ubuntu2604:~/xv6-riscv$ grep -n "case" user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
vboxuser@Ubuntu2604:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
vboxuser@Ubuntu2604:~/xv6-riscv$
```

Figure 4.1: Studying user/sh.c

1. **Name the five kinds of command the xv6 shell knows and the character that produces each.**
   xv6 knows the following commands: `EXEC` (a plain command, no special character), `REDIR` (`<`, `>`, `>>`), `PIPE` (`|`), `LIST` (`;`) and `BACK` (`&`). Parentheses only group commands.

2. **Where is cd handled, and why is it not run as a separate program?**
   `cd` is handled in `main` of `sh.c`, in the loop that reads input lines, and not in `runcmd`. It cannot be a separate program because programs run in a forked child, and the current directory is a property of the process. A child's `chdir` would change only the child's directory, so the shell itself must call `chdir`. That's why the earlier command failed.

3. **Search sh.c for any code that handles * or $. What does that explain about Part 2?**
   There is no code in `sh.c` for `*` or `$`. The xv6 shell does no globbing and no variable expansion, so these characters are passed unchanged to the program. This explains Part 2: `ls *` printed `ls: cannot open *` and `echo $HOME` printed `$HOME` instead of \home\vboxuser.

CONCLUSION

In this laboratory work I compared the xv6 shell with bash and studied the source code of `user/sh.c`.

In Part 1, the `sleep` program from Lab 2 had to be updated, because the xv6 authors renamed the system call `sleep()` to `pause()`. However I had already changed this back in Lab 2, so the program built and ran correctly.

In Part 2, the comparison showed that the xv6 shell is very small. It supports the basic mechanisms of a Unix shell: running programs, redirection (`<`, `>`), pipes (`|`), sequences (`;`), grouping with parentheses and background execution (`&`). Bash adds globbing, variable expansion, quoting, job control messages, a `PATH`, command history and tab completion. In xv6, `ls` stops working after `cd d` because there is no `PATH`: the shell looks for the program only in the current directory, while the programs live in the root directory. It can be run with a path such as `/ls` or `../ls`. Also, `cd` works only when it is the first command on the line, so `mkdir d ; cd d ; ls ..` fails with `exec cd failed`.

In Part 3, the `args` program showed that the program only receives the final list of arguments. For `args *.c "x y"`, xv6 passed 3 arguments after the program name (`*.c`, `"x`, `y"`), while bash passed 2 (`args_linux.c` and `x y`). So the shell, not the program or the kernel, expands `*.c`, removes quotes and replaces `$HOME`.

In Part 4, reading `sh.c` showed that the shell knows five kinds of command: `EXEC`, `REDIR`, `PIPE`, `LIST` and `BACK`. It contains no code for `*` or `$`, which explains the differences seen in Part 2. The command `cd` is handled in `main`, not in `runcmd`, because the current directory belongs to the process: a child process cannot change the directory of the shell.

Overall, the work showed that a shell is an ordinary user program, and that many features people associate with the “command line” (globbing, variables, quotes) belong to the shell and not to the operating system or to the programs.

REFERENCES

1. https://github.com/mit-pdos
2. https://pdos.csail.mit.edu/6.828/2023/xv6/book-riscv-rev3.pdf
3. https://www.geeksforgeeks.org/linux-unix/file-globbing-linux/
4. https://www.geeksforgeeks.org/linux-unix/difference-between-single-and-double-quotes-in-shell-script-and-linux/
5. https://www.geeksforgeeks.org/linux-unix/different-shells-in-linux/